

- [Complete External Storage & Server Deployment Guide](#)
 - [Document 1: Acasis Dual-Bay Server Architecture — Deployment Cheat Sheet](#)
 - [Document 2: Master Cheat Sheet — External Storage Provisioning & Maintenance](#)
 - [Document 3: Encrypted Multi-Drive Storage Vault — Complete Deployment & Recovery Blueprint](#)

Complete External Storage & Server Deployment Guide

Merged reference combining: (1) Acasis Dual-Bay HDD Server Setup, (2) Master External Storage Cheat Sheet (LUKS + Hardware Optimization), and (3) Encrypted Multi-Drive Storage Vault Blueprint. All topics from all three source documents are preserved below, in their original order, grouped by source document.

Document 1: Acasis Dual-Bay Server Architecture — Deployment Cheat Sheet

Copy-paste deployment cheat sheet for the Acasis dual-bay server architecture, meant to be executed in under 10 minutes once hardware arrives.

Phase 1: The Physical Boot-Up

1. Insert the Toshiba 1TB HDD into Slot 1 of the Acasis EC-6104 dock.
2. Plug the Acasis 36W power adapter into the wall.
3. Plug the Acasis USB cable into one of the Blue USB 3.0 ports on your Raspberry Pi.
4. Flip the power switch on the Acasis dock to ON.

Phase 2: Format & Motor Protection (hdparm)

Format the drive to Linux standards and permanently disable the sleep timer so the motor bearings stay warm and spinning.

1. Find the Drive Letter:

```
lsblk
```

(Look for your 1TB drive, it will likely be sdb or sdc. This guide uses sdb.)

2. Format the Drive:

```
sudo mkfs.ext4 /dev/sdb
```

3. Install the Motor Control Tool:

```
sudo apt update && sudo apt install hdparm -y
```

4. Lock the Spin State Permanently:

```
sudo nano /etc/hdparm.conf
```

Scroll to the absolute bottom of the file and paste this block to disable spindown:

```
/dev/sdb {  
    spindown_time = 0  
}
```

(Save and exit: Ctrl+O, Enter, Ctrl+X)

Phase 3: The Bulletproof Auto-Mount

If the power goes out in Dhaka, the Pi must automatically reconnect this drive when it reboots.

1. Create the Master Folder and Set Permissions:

```
sudo mkdir -p /mnt/server_storage
sudo chown -R redwannabil:redwannabil /mnt/server_storage
```

2. Get the Drive's Unique Serial Number (UUID):

```
sudo blkid /dev/sdb
```

(Copy the exact text inside the quotes after UUID=)

3. Inject the Auto-Mount Rule:

```
sudo nano /etc/fstab
```

Add this exact line to the very bottom of the file (replace YOUR-COPIED-UUID with your actual string):

```
UUID=YOUR-COPIED-UUID /mnt/server_storage ext4 defaults,noatime 0 2
```

(Save and exit: Ctrl+O, Enter, Ctrl+X)

4. Force the Mount Command:

```
sudo mount -a
```

Phase 4: Data Routing

Now that the 1TB drive is securely mounted to /mnt/server_storage, divide it into two operational zones.

1. Create the Sub-Directories:

```
mkdir /mnt/server_storage/cctv
mkdir /mnt/server_storage/nas
```

2. Reroute the CCTV System:

- Open MotionEye (or update your record_cctv_motion shell command).
- Change the recording destination directory to /mnt/server_storage/cctv.

3. Reroute the Docker NAS Services:

- Open your docker-compose.yml file for Nextcloud and Filebrowser.
- Change the storage volume mapping from the old drive path to /mnt/server_storage/nas.
- Apply the changes by running:

```
sudo docker compose up -d
```

Result: The Pi becomes a dual-purpose NAS and NVR server that avoids chewing through flash memory or wearing out its internal motors. Next step referenced: configuring the Tapo camera side of the network.

Document 2: Master Cheat Sheet — External Storage Provisioning & Maintenance

A-to-Z master cheat sheet covering drive provisioning, encryption, hardware-specific optimization, network sharing, and emergency recovery, tuned for Raspberry Pi and Docker.

Phase 1: Drive Identification & LUKS Encryption

Always format and encrypt drives manually to prevent OS partition leaks.

1. Identify the Drive:

```
lsblk
# Note the target drive, e.g., /dev/sdX1
```

2. Generate a Secure 64-Bit Native Key:

```
sudo mkdir -p /root/.keys
sudo chmod 700 /root/.keys
sudo dd if=/dev/urandom of=/root/.keys/newdrive.key bs=64 count=1
sudo chmod 600 /root/.keys/newdrive.key
```

3. Format and Open the LUKS Vault:

```
sudo cryptsetup luksFormat --type luks2 -d /root/.keys/newdrive.key /dev/sdX1
sudo cryptsetup luksOpen -d /root/.keys/newdrive.key /dev/sdX1 newdrive_vault
```

4. Create the ext4 Filesystem:

```
sudo mkfs.ext4 /dev/mapper/newdrive_vault
```

Phase 2: Persistent UUID Mounting

Locking drives via UUID ensures they never mount to the OS NVMe if a dock reconnects.

1. Get the UUID:

```
sudo blkid
# Copy the UUID for the raw partition (e.g., /dev/sdX1)
```

2. Configure /etc/crypttab (Unlocking):

> Note: Never use the nofail flag in this file — it causes boot errors.

```
sudo nano /etc/crypttab
# Add this line:
newdrive_vault UUID=YOUR-UUID-HERE /root/.keys/newdrive.key luks,discard
```

3. Configure /etc/fstab (Mounting):

> Note: nofail belongs here so the Pi boots even if the drive is unplugged.

```
sudo mkdir -p /mnt/newdrive_storage
sudo nano /etc/fstab
# Add this line:
/dev/mapper/newdrive_vault /mnt/newdrive_storage ext4 defaults,nofail 0 2
```

4. Apply and Verify:

```
sudo systemctl daemon-reload
sudo mount -a
df -h
```

Phase 3: Hardware Armor (Orico Dock & SMR Drive Optimizations)

Essential for preventing JMicron chip lockups and SMR cache exhaustion during massive 40GB+ transfers.

1. Apply the 180-Second SCSI Timeout Rule

— prevents the Linux kernel from dropping the drive when SMR heads stall to repair bad sectors:

```
echo 'ACTION=="add", SUBSYSTEM=="scsi", ENV{DEVTYPE}=="scsi_device",
ATTR{device/timeout}="180"' | sudo tee /etc/udev/rules.d/99-usb-timeout.rules
sudo udevadm control --reload-rules
sudo udevadm trigger
```

2. **Apply the 15MB/s Write Straitjacket (cgroups)** — prevents the JMicron USB chip from overheating by throttling sequential write blasts:

```
# Convert the mount path to a systemd unit name (e.g., mnt-newdrive_storage.mount)
sudo systemctl set-property mnt-newdrive_storage.mount
IOWriteBandwidthMax=/dev/mapper/newdrive_vault 15M
```

Phase 4: Ultimate Permissions (Samba + Nextcloud Synergy)

Unifies read/write/delete access so Windows File Explorer and Docker Nextcloud do not fight over directory ownership.

1. **Make the Users Share Groups:**

```
sudo usermod -aG www-data redwannabil
sudo usermod -aG redwannabil www-data
```

2. **Force Co-Ownership and 775 Permissions:**

```
# Assign the owner to redwannabil and the group to www-data
sudo chown -R redwannabil:www-data /mnt/newdrive_storage
# Grant full Read(4)+Write(2)+Execute/Delete(1) to both Owner and Group (775)
sudo chmod -R 775 /mnt/newdrive_storage
```

Phase 5: Network Mapping

1. **Add to Samba (/etc/samba/smb.conf)** — create an explicit block; do not browse through PiMaster in Windows:

```
[New_Drive]
comment = New Encrypted Storage
path = /mnt/newdrive_storage
writeable = yes
force user = redwannabil
browseable = yes
guest ok = no
create mask = 0777
directory mask = 0777
```

Restart Samba:

```
sudo systemctl restart smbd nmbd
```

2. **Add to Nextcloud (Docker):** In the Nextcloud Web UI, go to **Administrative Settings → External Storage**.

- Folder Name: New_Drive
- External Storage Type: Local
- Configuration (Path): /mnt/newdrive_storage
- Click the checkmark to save.

Phase 6: Monitoring Diagnostics

Watch real-time hardware performance in SSH/PowerShell to verify files are writing to the physical disks (wMB/s) and not leaking to the OS NVMe (nvme0n1).

```
# Refreshes every 2 seconds in Megabytes
iostat -x -m 2
```

- **wMB/s:** Shows actual write speed (watch for the 15MB/s limit).
- **w_await:** If this spikes over 1500ms, the SMR drive has hit a bad sector and is repairing itself — the 180s timeout rule is actively keeping it alive.

Phase 7: Emergency Recovery & Maintenance

If a dying SMR drive (like cctv_hdd) drops offline, causes an Input/output error, or gets

locked in a Read-only file system state, run this block to wake it up instantly.

The “Wake-Up” Script:

```
# 1. Force unmount the stuck directory
sudo umount -l /mnt/cctv_hdd
# 2. Close the broken encryption channel
sudo cryptsetup luksClose cctv_hdd
# 3. Restart the crypto disk from /etc/crypttab
sudo cryptdisks_start cctv_hdd
# 4. Remount all drives from /etc/fstab
sudo mount -a
```

(Replace cctv_hdd with the specific drive name if a different drive crashes.)

The Nextcloud Synchronization Command: Anytime you manually copy files into a drive via Windows/Samba, Nextcloud’s database won’t know they exist. Run this exact Docker command to force Nextcloud to rescan everything and update the web UI:

```
sudo docker exec --user www-data nextcloud-app-1 php occ files:scan --all
```

Document 3: Encrypted Multi-Drive Storage Vault — Complete Deployment & Recovery Blueprint

A-to-Z deployment and recovery blueprint for an encrypted multi-drive storage vault, combining drive encryption, storage pooling, Docker container integration, startup automation, backup safety, and emergency data rescue.

Hardware Inventory & Strategy Review

- **Physical Drives:** 1× 256GB Pen Drive, 1× 120GB SSD, 1× 1TB HDD, 1× 512GB HDD.
- **Security Standard:** LUKS (Linux Unified Key Setup) AES-256 operating-system-level encryption.
- **Pooling Layer:** MergerFS fuses all drives into a single 1.88TB virtual pool (/mnt/storage_pool).
- **Container Access:** Applications (Immich, Nextcloud, Filebrowser, Samba, Home Assistant) access data paths inside the decrypted pool.

⚠ **CRITICAL PERMANENT DATA LOSS WARNING:** Encrypting a hard drive completely wipes its contents. Since photos are currently on these drives, copy all photos off the external drives to an external backup or laptop before proceeding. Do not run Phase 2 on a drive until its files are safely backed up.

Phase 1: Cryptographic Backup & Passphrase Keys

A strict separation of keys is enforced: an internal Keyfile handles automatic mounts on system startup, while a Printed Paper Wallet Passphrase protects the master override.

1. **Generate Your Printed Master Passphrase:** Run this command on your laptop or server to generate a secure, random text passphrase string:

```
openssl rand -base64 48
```

Action Required: Print or write down this exact generated string on a physical piece of paper (your Paper Wallet). Store it in a secure drawer or physical safe. Never save this text file anywhere online or on a device connected to the internet.

Phase 2: OS Partitioning, Encryption, and Formatting

Plug all 4 external storage drives into the USB ports of your Raspberry Pi 5.

Step 1: Install System Dependencies

```
sudo apt update
sudo apt install cryptsetup mergerfs -y
```

Step 2: Identify Drive Identifiers

```
lsblk
```

Note down the device letters (e.g., /dev/sda, /dev/sdb, /dev/sdc, /dev/sdd). /dev/sda is used as the standard baseline below; repeat this step for each drive letter.

Step 3: Wipe, Encrypt, and Setup File Systems Execute this exact sequence for all 4 drives consecutively, substituting sda with your targeted drive letters, and updating the mapper destination index (disk1, disk2, etc.) respectively.

1. **Format with LUKS Encryption** — create the AES-256 boundary block; enter your master printed passphrase when prompted:

```
sudo cryptsetup luksFormat /dev/sda
```

2. **Open the Encrypted Tunnel** — expose the secure layer to the OS kernel under a mapping label:

```
sudo cryptsetup luksOpen /dev/sda disk1
```

3. **Format with EXT4 File System** — write a native Linux EXT4 journal framework onto the mapped crypto-container:

```
sudo mkfs.ext4 /dev/mapper/disk1
```

4. **Establish Mount Directories** — make local directories on the OS filesystem to host the independent drives:

```
sudo mkdir -p /mnt/disk1
sudo mount /dev/mapper/disk1 /mnt/disk1
```

(Complete this loop for all remaining devices: map /dev/sdb → disk2 mounted at /mnt/disk2, /dev/sdc → disk3 mounted at /mnt/disk3, and /dev/sdd → disk4 mounted at /mnt/disk4.)

Phase 3: The MergerFS Storage Pool Virtualization

With all 4 disks independently decrypted and mounted, fuse them into a single file pool directory tree.

```
# 1. Create the master pooling directory
sudo mkdir -p /mnt/storage_pool

# 2. Fuse the disks using MergerFS
sudo mergerfs /mnt/disk1:/mnt/disk2:/mnt/disk3:/mnt/disk4 /mnt/storage_pool -o
use_ino,allow_other,minfreespace=50G,fsname=mergerfs
```

- use_ino: Keeps system file descriptors accurate for Docker environments.
- allow_other: Permits containerized application profiles to write inside the shared space.
- minfreespace=50G: Dynamically moves files away from drives getting within 50GB of capacity limits to shield them from file errors.

```
# 3. Create structured spaces for application runtimes
sudo mkdir -p /mnt/storage_pool/immich_data
sudo mkdir -p /mnt/storage_pool/nextcloud_data
sudo mkdir -p /mnt/storage_pool/filebrowser_data
sudo mkdir -p /mnt/storage_pool/samba_shares

# 4. Correct permissions to match the default Docker uid/gid access values
sudo chown -R 1000:1000 /mnt/storage_pool/*
```

Phase 4: Full Automation of Boot Decoding

To prevent manual command entry after power grid fluctuations or system updates, generate an automated system boot script anchored to a highly protected hardware key file.

Step 1: Provision and Secure the Local Keyfile

```
sudo mkdir -p /root/.keys
sudo openssl rand -out /root/.keys/vault.key 512
sudo chmod 400 /root/.keys/vault.key
```

Setting permissions to 400 locks this key file down so that only the root user kernel can read its contents.

Step 2: Inject the Keyfile into Your Encryption Headers Instruct the encrypted drives to accept either the master paper wallet passphrase or this internal system keyfile:

```
sudo cryptsetup luksAddKey /dev/sda /root/.keys/vault.key
sudo cryptsetup luksAddKey /dev/sdb /root/.keys/vault.key
sudo cryptsetup luksAddKey /dev/sdc /root/.keys/vault.key
sudo cryptsetup luksAddKey /dev/sdd /root/.keys/vault.key
```

(Type the master paper wallet passphrase to authorize each addition.)

Step 3: Establish the Auto-Mount Initialization Script Create the system execution file:

```
sudo nano /usr/local/bin/mount-vault.sh
```

Paste this complete initialization sequence inside:

```
#!/bin/bash
### BEGIN INIT INFO
# Provides: mount-vault
# Required-Start: $local_fs
# Default-Start: 2 3 4 5
# Default-Stop: 0 1 6
### END INIT INFO

# 1. Unveil the cryptographic mapping layer using the localized key
cryptsetup luksOpen /dev/sda disk1 --key-file /root/.keys/vault.key
cryptsetup luksOpen /dev/sdb disk2 --key-file /root/.keys/vault.key
cryptsetup luksOpen /dev/sdc disk3 --key-file /root/.keys/vault.key
cryptsetup luksOpen /dev/sdd disk4 --key-file /root/.keys/vault.key

# 2. Attach physical drives to system partition directories
mount /dev/mapper/disk1 /mnt/disk1
mount /dev/mapper/disk2 /mnt/disk2
mount /dev/mapper/disk3 /mnt/disk3
mount /dev/mapper/disk4 /mnt/disk4

# 3. Restructure the virtual data pool mapping layer
mergerfs /mnt/disk1:/mnt/disk2:/mnt/disk3:/mnt/disk4 /mnt/storage_pool -o
use_ino,allow_other,minfreespace=50G,fsname=mergerfs

# 4. Wake up Docker instances now that paths are ready
systemctl restart docker
```

Save and close (Ctrl+O, Enter, Ctrl+X).

```
# Make the service script executable
sudo chmod +x /usr/local/bin/mount-vault.sh
# Register the execution service into the system init runlevels
sudo update-rc.d mount-vault.sh defaults
```

Phase 5: Container Application Path Integrations

Update system service application paths to use the encrypted pool folders instead of the internal OS SSD drive.

1. Immich Configuration Alignment

```
cd ~/homelab/immich
docker compose down
nano .env
```

Modify the environment variable string to target the pool destination path:

```
UPLOAD_LOCATION=/mnt/storage_pool/immich_data
```

Save the file, open the core configuration compose script (nano docker-compose.yml), ensure resource settings match the values below to lock down the system against SSD swap wear errors, and start the app:

```
services:
  immich-server:
    container_name: immich_server
    # ... mapping values ...
    deploy:
      resources:
        limits:
          memory: 600M # Keeps container from choking or thrashing swap
  redis:
    container_name: immich_redis
    deploy:
      resources:
        limits:
          memory: 64M
  database:
    container_name: immich_postgres
    deploy:
      resources:
        limits:
          memory: 256M
```

```
docker compose up -d
```

2. Nextcloud & Filebrowser Integrations Update the volume configuration maps within their respective compose environment scripts:

```
# Inside Nextcloud docker-compose.yml configuration
volumes:
  - /mnt/storage_pool/nextcloud_data:/var/www/html/data

# Inside Filebrowser docker-compose.yml configuration
volumes:
  - /mnt/storage_pool/filebrowser_data:/srv
```

3. Samba Networking Shares File Adjustment Update the internal system network share service profiles:

```
sudo nano /etc/samba/smb.conf
```

Update the directory paths to target the pool:

```
[SecureShare]
comment = Encrypted Home Network Storage
path = /mnt/storage_pool/samba_shares
browseable = yes
read only = no
guest ok = no
```

```
sudo systemctl restart smbd
```

Phase 6: Automated Memory & Storage Optimization

Preserve system health and lock down memory channels against unexpected drive writes by registering a system memory cleanup sequence.

Step 1: Create the Automated Cleanup Script

```
nano ~/autoclean.sh
```

Paste this complete system optimization block inside:

```
#!/bin/bash
# 1. Purge dead dependencies and upgrade installations
apt autoremove -y
apt clean

# 2. Reclaim logged journal space older than 3 days
journalctl --vacuum-time=3d

# 3. Clean up orphaned Docker engine image builds
docker system prune -a -f

# 4. Clear root admin temp files
rm -rf /home/redwannabil/.cache/*
```

Save and close (Ctrl+O, Enter, Ctrl+X). Then make it executable:

```
chmod +x ~/autoclean.sh
```

Step 2: Establish the 2:30 AM Cron Automation Open the primary root automation task tracker list:

```
sudo crontab -e
```

Add this precise operational instruction sequence line at the bottom:

```
30 2 * * * /home/redwannabil/autoclean.sh > /home/redwannabil/autoclean.log 2>&1
```

Phase 7: Emergency Recovery Procedures

If the system encounters a hardware issue, this section shows how to rescue files on any machine using the printed master passphrase.

Recovery Scenario A: Extracting Files on Another Linux Computer 1. Plug any of the encrypted drives into a recovery machine's USB port. 2. Open a terminal instance and access the encrypted content array: `bash sudo cryptsetup luksOpen /dev/sda rescue-disk` 3. Type the master backup password string from the physical printed paper wallet. 4. Mount the partition to access the files directly: `bash sudo mkdir -p /mnt/recovery sudo mount /dev/mapper/rescue-disk /mnt/recovery` The photos will instantly appear inside /mnt/recovery, fully accessible and clean.

Recovery Scenario B: Extracting Files on a Windows Computer 1. Download and install the open-source filesystem extraction tool LibreCrypt or VeraCrypt (with LUKS compilation support enabled). 2. Plug the encrypted external drive into the Windows machine's USB slot. 3. Open the software, click **Open Container / Device**, and select the target USB device letter assignment. 4. Provide the master password string from the physical paper wallet. 5. The software tool will safely map the partition window to a traditional drive volume identifier index (like E: or F:), allowing interaction with the photos inside Windows File Explorer.